# MS2 Summary: Lake Geneva Water Temperature Forecast

Maria Micioni, MLOps HS26 (I.BA_MLOPS) · Repository: github.com/mariamicioni/lake-geneva-temp · 01.11.2026

## 1 What I achieved in this milestone

My feature pipeline now runs on its own every evening. It downloads new water data and weather forecasts, builds the feature table again and saves a new data version in my Google Cloud Storage bucket.

- **Download and backfill.** The water data (temperature and flow of station 2606, every 10 minutes) comes from the BAFU Data Platform. If BAFU does not answer, the script uses existenz.ch. The same scripts can also download old data for any dates: water data back to 2016 (569,280 readings, every day has at least 120 of 144), the 48-hour-old Open-Meteo forecast back to 2022 (all variables from 20.01.2024) and the 06:00 UTC ECMWF forecast runs from 10.08.2024 (808 runs).
- **One feature definition.** The file `src/lakegeneva/common/features.py` makes one row per day (Swiss time). It has five water features from the readings before 18:00, five weather features (two of them are the new south-west wind) and the label: the mean temperature of the next day, if that day is over and at least 120 of its 144 readings exist. Training and the live forecast will use the same file. A test checks that readings after 18:00 change no feature.
- **Feature store.** The table is saved as a Parquet file. DVC keeps its versions in the bucket, and Git only stores small pointer files. `data-v1` is the first version, and the daily job now saves one new version every day. I tested it: a new clone plus `dvc pull` gave back exactly the same files.
- **Schedule, Docker and tests.** The workflow `.github/workflows/feature-pipeline.yml` runs every day at 19:00 UTC. The first run on 27.10 took 48 seconds, and the first scheduled run that night worked too. CI runs 62 tests and builds the Docker image (389 MB, runs without root rights). The README and `DATACARD.md` explain how to run each step and what each column means.

**Key decisions.**

- I use the weather forecast that was made 48 hours earlier, for training and for the live forecast. This forecast always existed before 18:00, also when GitHub starts the job late (in October it started between 19:33 and 00:36). The downside is that this forecast is older. So I also save the 06:00 UTC forecast run, which is ready before 18:00 too, and I will compare both in MS3.
- The live water data now comes from BAFU, because the old data comes from BAFU too. So training and the live forecast use the same data.
- BAFU publishes its values about 1.5 hours late, so a forecast cannot really go out at 18:00 as my proposal says (a0973be). I keep 18:00 as the cutoff for the data, and the forecast is ready after the evening job (19:00 UTC at the earliest). A forecast at 18:00 sharp would only have the readings until about 16:30, and persistence would be about 4 % worse.
- All raw data is saved in UTC time, and only `features.py` changes it to Swiss time (with summer time).

**Problems and limits.** Sunshine and wind in the old forecasts only start on 20.01.2024. So the full model has 1,009 rows, not 3,949 like the water data. I found two time problems late. The old download job never ran before 18:00. And Open-Meteo's Swiss time moves all hours by the same offset, so in winter even summer hours were one hour wrong. I fixed both (dd3919c, 1a70d28, eb3bb2a, d5af820), and the effect on the saved data was small. I also fixed a label that was set before the next day was over (3b95863), and raw files that a second run on the same day replaced (cbef321). The Windows setup also took time. One `.gitignore` rule made DVC skip the data folder, my antivirus blocked DVC's internet connection until I added a certificate file, and I had to update Docker Desktop.

**Where to look first.** `src/lakegeneva/common/features.py` and `tests/test_features.py`, the "Feature pipeline" runs in the Actions tab (with the daily commits "chore(data): daily update"), and the part "Goal and success criterion" in the README. You can run it without a key: `docker build -t lake-geneva-features .` and then `docker run --rm lake-geneva-features`. This downloads the last 7 days and builds a small table from them. The full table is in my private bucket, but the "Build the feature table" step in each run's log shows its size (3,956 days on 29.10). Without a key, the backfill commands in the README rebuild the whole table from the public APIs in about 11 minutes.

## 2 Response to the MS1 reviews

The reviewers read my ILIAS proposal from 30.09. My repository version from 02.10 (4fc4474) had already changed some points, and MS2 changed more:

| | ILIAS proposal (30.09) | Now (MS2) |
|---|---|---|
| History | 32 days from existenz.ch, about 90 rows by MS3 | BAFU 10-minute data since 2016: 1,009 rows with all features, 3,949 rows with the water features |
| Baseline | persistence 0.42 °C on 15 pairs of days in August and September | 0.53 °C at 18:00 on the 643 test days (August and September over ten years: 0.69 °C, so the 15 pairs were too optimistic) |

| | | |
|---|---|---|
| Target | MAE at most 0.33 °C | at least 20 % lower MAE than the stronger baseline, on the same days: 0.425 °C or less on the test days (A2/B2, B3) |
| Tools | W&B, GitHub Release | DVC on Google Cloud Storage; MLflow on DagsHub planned for MS3 |
| Forecast data | Single Runs for the backfill | 48-hour-old forecast for training and live; Single Runs only to compare |
| Live water data | existenz.ch | BAFU Data Platform, existenz.ch as backup |

**1. A1 + B1, too few rows: Partly** (the history is changed, the evaluation comes in MS3). The history now comes from the BAFU Data Platform (049452d, 165e8cf). It has the same BAFU data as the page the editors linked, but I can download it myself through an open API. Together with the 48-hour-old forecasts (dd3919c), this gives 1,009 rows with all features, from 21.01.2024 to 25.10.2026. The split in rows: validation 366 rows (21.01.2024 to 20.01.2025), to choose the model settings with growing time windows. Test 643 rows (21.01.2025 to 25.10.2026). Training: for each test day, the model is trained again on all rows before it (walk-forward, MS3), so the training grows from 366 to 1,008 rows. 159 test days are in October to January, and 100 test days have a change of more than 0.8 °C. The bootstrap for the confidence interval of the MAE difference (d4b354c) and the start of the test days are in `src/lakegeneva/common/baselines.py`.

**2. A2 + B2, persistence is biased when the lake cools: A2 Changed, B2 Partly.** I added trend-adjusted persistence (the last reading before 18:00 plus a third of `water_trend_3d`) in `baselines.py` (d4b354c). On my data it is worse than normal persistence: 0.722 against 0.532 °C on the 643 test days (difference +0.19 °C, 95 % interval +0.13 to +0.26), and 0.334 against 0.258 °C on all October-to-January days since 2016 (+0.05 to +0.11). On the 159 October-to-January test days they are about the same (0.279 against 0.276, interval −0.04 to +0.07). The lake does cool, but the daily mean drops by only 0.08 °C per day, while it jumps by 0.54 °C from day to day (standard deviation). So adding the trend adds more error than it removes, and normal persistence is the stronger baseline. My target is now relative: at least 20 % lower MAE than the stronger baseline on the same days, so 0.425 °C or less on the test days. A fixed number does not work, because the MAE of persistence goes from 0.18 to 0.35 °C between October-to-January seasons. B2 also says that a pass may not show that the weather helps. This part is **Not yet** done: in MS3 I will train a model with only the water features next to the full model.

**3. E1, originality: Changed.** My proposal missed the newer KTH ID2223 list. I left the proposal file as it is. Instead, the README part "Related work" (bb28918) now names the KTH ID2223 2025/26 project "Water Temperature at Södertälje" (bathing sites, open data from the city and Open-Meteo) and the HS26 project that forecasts river temperature with existenz.ch and Open-Meteo. My project is different because it is the outflow of a large lake. Persistence is very strong there. But in summer the lake is layered (stratified): warm water on top, cold water below. South-west wind pushes the warm water away from Geneva, and the cold water comes up (upwelling). So 527 of the 593 days with a change of more than 0.8 °C since 2016 are in April to September. For this reason I added the south-west wind features and the target for days with big changes. A comparison with Alplakes stays optional for MS4.

**4. A3, wind direction: Changed.** The saved forecasts now include the wind direction (e8ec9f2, dd3919c). `features.py` has two new features from the same 48-hour-old forecast as the other weather features: `wind_sw_fc_d0` and `wind_sw_fc_d1`. They are the mean of speed × cos(direction − 225°) over the hours of today and of tomorrow (7130889), so they measure how much the wind blows from the south-west. From April to September, this south-west wind is linked twice as strongly to tomorrow's change as the maximum wind speed (correlation −0.29 against −0.13). One example: with south-west wind on 6 and 7 July 2025, the water cooled from 26.0 °C (5 July) to 14.5 °C (8 July).

**5. B3, days with big changes in the success criterion: Changed.** The success criterion now has its own target for days with a change of more than 0.8 °C (README, bb28918). 100 of my test days are such days. On them, persistence has an MAE of 1.921 °C, so the model must reach 1.537 °C or less. 100 days are enough to judge this: on these days, the 95 % interval of the difference between the two baselines is only about ±0.18 °C, while 20 % means 0.38 °C. October to January alone has only 6 such test days. That is too few for a target, so I only report it.

**6. E2, one tool for tracking and registry: Changed** (4fc4474, already before the reviews). The proposal, the README and `.env.example` only name MLflow on DagsHub, and W&B is removed everywhere. Why: the course uses MLflow, and DagsHub hosts it for free for public repositories, so I do not run my own server. Its registry alias "champion" marks the model that is served. For the data, DVC replaced the GitHub Release, so every data version is a Git commit and every model can name its data. The MLflow code comes in MS3.